

Python functions call by reference [duplicate]

Asked 11 years, 6 months ago Modified 1 year, 4 months ago Viewed 204k times

109

This question already has answers here:
[How do I pass a variable by reference?](#) (41 answers)

A community member has associated this post with a similar question.

Closed 1 year ago by [Karim Knechtel](#) · [python](#)

(List of close votes is only viewable by users with the close/reopen votes privilege)

Edit question

In some languages you can pass a parameter by reference or value by using a special reserved word like `ref` or `val`. When you pass a parameter to a Python function it never alters the value of the parameter on leaving the function. The only way to do this is by using the `global` reserved word (or as I understand it currently).

Example 1:

```
k = 2

def foo(n):
    n = n + n    #clarity: reporting comment below
    square = n
    return square

j = foo(k)
print j
print k
```

would show

```
>>>4
>>>2
```

showing k to be unchanged.

In this example the variable n is never changed

Example 2:

```
n = 0
def foo():
    global n
    n = n + n
    return n
```

In this example the variable n is changed.

Is there any way in Python to call a function and tell Python that the parameter is either a value or reference parameter instead of using global?

python

variables

reference

pass-by-reference

Edit tags

Share

Edit

Follow

Acopen

Delete

Flag

Unprotect

edited Aug 14, 2012 at 21:57

asked Nov 8, 2012 at 23:01

 31.9k 26 122 185

 2,244 6 25 34

5  The best resource I've found for understanding python's calling model is this article on effort: [effbot.org/zone/call-by-object.htm](#) – Wikiduck Nov 8, 2012 at 23:04

1  you should read about python variables: mutable and immutable objects. Also in your first example why would `n` be changed, you are using a new variable `square` to store the result of your calculation. – Fernando Casco Nov 8, 2012 at 23:07

2  Also, have a look at this previous StackOverflow answer: [stackoverflow.com/q/10262945/173292](#) It explains the call by object reference model fairly intuitively. – Wikiduck Nov 8, 2012 at 23:41

1  See also this SO question for very useful explanation: [how do I pass a variable by reference](#) – Michael Ohlroge Sep 13, 2016 at 14:22

 I have a work around to pass a to function(s) by reference: [functools.partial](#), how the data in list is a reference. I'm not posting as an answer because this seems like the wrong way but I'd love to be told the correct way to do this. – Supamee Jul 19, 2017 at 16:04

15 Answers

220

There are essentially three kinds of 'function calls':

- Pass by value
- Pass by reference
- Pass by object reference

Python is a PASS-BY-OBJECT-REFERENCE programming language.

Firstly, it is important to understand that a variable, and the value of the variable (the object) are two separate things. The variable 'points to' the object. The variable is not the object. Again:

THE VARIABLE IS NOT THE OBJECT

Example: in the following line of code:

```
>>> x = []
```

[] is the empty list, `x` is a variable that points to the empty list, but `x` itself is not the empty list.

Consider the variable (`x` in the above case) as a box, and 'the value' of the variable (`[]`) as the object inside the box.

PASS BY OBJECT REFERENCE (Case in python):

Here, "Object references are passed by value."

```
def append_one(l1):
    l1.append(1)
    x = [8]
    append_one(x)
    print x
```

Here, the statement `x = [8]` makes a variable `x` (box) that points towards the object `[8]`.

On the function being called, a new box `x1` is created. The contents of `x1` are the SAME as the contents of the box `x`. **Both the boxes contain the same object.** That is, both the variables point to the same object in memory. Hence, any change to the object pointed at by `x1` will also be reflected by the object pointed at by `x`.

In conclusion, the output of the above program will be:

```
[8, 1]
```

Note:

If the variable `x1` is reassigned in the function, then `x1` will point to a separate object in memory. `x` however, will continue pointing to the same object in memory it was pointing to earlier.

Example:

```
def append_one(l1):
    l1 = [8, 1]
    x = [8]
    append_one(x)
    print x
```

The output of the program will be:

```
[8]
```

PASS BY REFERENCE:

The box from the calling function is passed on to the called function. Implicitly, the contents of the box (the value of the variable) is passed on to the called function. Hence, any change to the contents of the box in the called function will be reflected in the calling function.

PASS BY VALUE:

A new box is created in the called function, and copies of contents of the box from the calling function is stored into the new boxes.

Share

Edit

Follow

Flag

edited Dec 23, 2012 at 7:53

answered Oct 11, 2015 at 15:15

 17.1k 15 112 394

 2,232 2 10 17

4  Could you give examples of "pass by reference" and "pass by value" like you did for "pass by object reference"? – Tsai Oct 26, 2017 at 0:41

 @Tsai Pass by reference and pass by value can be seen in C++. This answer gives a good explanation: [stackoverflow.com/a/430958/5075583](#) – Shubhi Verma Oct 27, 2017 at 2:51

6  I still don't understand how this differs from pass-by-reference. This answer desperately needs an example of how identical pseudocode would produce different outputs for all 3 paradigms. Linking to a different answer which only explains the difference between pass-by-reference and pass-by-value doesn't help. – Jess Rodell Jan 22, 2018 at 0:02

 Is it not more simple to say that "all variables in Python are pointers" and "all variables assigned to parameters in a function call are assigned to new pointers"? If you realize this, you'll see that Python is like Java. It simply always pass pointers by value. See [Kirk Strauser's answer](#) – Marco Sula Jan 1, 2020 at 10:55

 @MarcoSula no. Because Python doesn't have pointers. This is a silly Javasm, necessitated by the fact that Java has "primitive types" and "reference types". Such a distinction doesn't exist. – jumpstartultra Nov 3, 2021 at 16:48

10

Technically python do not pass arguments by value: all by reference. But ... since python has two types of objects: immutable and mutable, here is what happens:

- **Immutable arguments are effectively passed by value:** string, integer, tuple are all immutable object types. While they are technically "passed by reference" (like all parameters), since you can't change them in-place inside the function it looks/behaves as if it is passed by value.
- **Mutable arguments are effectively passed by reference:** lists or dictionaries are passed by their pointers. Any in-place change inside the function (like `append` or `del`) will affect the original object.

This is how Python is designed: no copies and all are passed by reference. You can explicitly pass a copy.

```
def sort(array):
    # do sort
    return array

data = [1, 2, 3]
sort(data) # here you passed a copy
```

Last point I would like to mention which is a function has its own scope.

```
def do_any_stuff_to_these_objects(x, y):
    a = x + 2
    del y['last_name']

number = 1 # immutable
hashmap = {'first_name': 'John', 'last_name': 'legend'} # mutable
do_any_stuff_to_these_objects(number, hashmap)
print(number) # 1, oh, it should be 2 ! no x is changed inside the function scope
print(hashmap) # {'first_name': 'John'}
```

Share

Edit

Follow

Flag

edited Sep 5, 2020 at 15:30

answered Apr 1, 2020 at 20:16

 4,766 7 35 37

 578 6 20

1  "Immutable arguments are effectively passed by value: string, integer, tuple are all passed by reference" this sentence has self-contradiction – eral Apr 17, 2020 at 18:33

 @eral, if you understand the full context, it does not contradict itself. I edited it to make this more clear. – Walek02 Sep 5, 2020 at 15:31

```
class DemoClass:
    x = 4
    y = 3
    foo1 = DemoClass()
    foo1.x = 2
    foo2 = DemoClass()
    foo2.y = 5

    def MySquare(self):
        myObj.x = myObj.y**2
        myObj.y = myObj.x**2
        print("foo1.x *", foo1.x)
        print("foo1.y *", foo1.y)
        print("foo2.x *", foo2.x)
        print("foo2.y *", foo2.y)
        mySquare(foo1)
        mySquare(foo2)
        print("After Square:")
        print("foo1.x *", foo1.x)
        print("foo1.y *", foo1.y)
        print("foo2.x *", foo2.x)
        print("foo2.y *", foo2.y)
```

Share

Edit

Follow

Flag

answered Aug 24, 2020 at 19:59

 11 1

Consider that the variable is a box and the value it points to is the "thing" inside the box:

1

1. **Pass by reference** : function shares the same box and thereby the thing inside also.

2. **Pass by value** : function creates a new box, a replica of the old one, including a copy of whatever thing is inside it. Eg. **Java** - functions create a copy of the box and the thing inside it which can be: a primitive / a reference to an object. (note that the copied reference in the new box and the original both still point to the same object, here the reference is the thing inside the box, not the object it is pointing to)

3. **Pass by object-reference**: the function creates a box, but it encloses the same thing the initial box was enclosing. So in **Python**:

a) if the thing inside said box is **mutable**, changes made will reflect back in the original box (eg. lists)

b) if the thing is **immutable** (like python strings and numeric types), then the box inside the function will hold the same thing UNTIL you try to change its value. Once changed, the thing in the function's box is a totally new thing compared to the original one. Hence **id()** for that box will now give the identity of the new thing it encloses.

Share

Edit

Follow

Flag

answered Feb 20, 2020 at 5:16

 11 3

You can not change an immutable object, like `str` or `tuple`, inside a function in Python, but you can do things like:

```
def foo(y):
    y[0] = y[0]**2

x = [5]
foo(x)
print x[0] # prints 25
```

That is a weird way to go about it, however, unless you need to always square certain elements in an array.

Note that in Python, you can also return more than one value, making some of the use cases for pass by reference less important:

```
def foo(x, y):
    return x**2, y**2

a = 2
b = 3
a, b = foo(a, b) # a == 4; b == 9
```

<https://stackoverflow.com/questions/13209427/python-functions-call-by-reference>

13

When you return values like that, they are being returned as a Tuple which is in turn unpacked:

edit: Another way to think about this is that, while you can't explicitly pass variables by reference in Python, you can modify the properties of objects that were passed in. In my example (and others) you can modify members of the list that was passed in. You would not, however, be able to reassign the passed in variable entirely. For instance, see the following two pieces of code look like they might do something similar, but end up with different results:

```
def clear_x(x):
    x = []

def clear_y(y):
    while y:
        y.pop()

x = [1,2,3]
clear_x(x) # x will not be changed
clear_y(y) # y will be emptied
```

Share Edit Follow Flag

edited Feb 19, 2020 at 23:00
Saman
316 2 11

answered Nov 8, 2012 at 23:13
dave markoff
17.6k 7 51 66

2 It would be helpful to see the results of the programs. For example does 'print x()' return 25? (yes it does, but that should be shown) – Ray Salemi Sep 3, 2017 at 17:13

12 @Babsey: `clear_x` is given a reference to `x` and does that reference in `x`. It then immediately changes `x` to reference a different array. The original `x` string is forgotten about in the scope of `clear_x`, but is not actually changed. It continues unchanged when returning to the global scope. Contrast that with `clear_y` which takes a reference to `x` and then operates directly on it, without ever creating a new array or otherwise pointing `x` and something different. – Dave Markoff Sep 8, 2017 at 14:21

`x` thanks dave I tried "creating a new array" in `clear_y`, `y = x`, `y.pop()` and then called `clear_y(x)`, `x` still got emptied... So we need something like `y = list(x)` to create a copy of `x` (list) explained [here](#) – alexy Sep 11, 2017 at 18:59

`run_thread = (True) <= threading.Thread(target=do_something, args=(1,1,run_thread)) --do something -- and when you want to see how thread() works -> Ask Python Feb 24, 2019 at 11:08`

1 Note that in Python, you can also return more than one value, making some of the use cases for pass by reference less important... Well not quite, as sometimes passing by reference has to do with memory management, not just with changing arguments values. Sometimes you don't want your arguments to be copied inside the routine. Specially if you are running short of memory. – user110309 Feb 17, 2020 at 11:35

Python is neither pass-by-value nor pass-by-reference. It's more of "object references are passed by value" as described [here](#)

1. Here's why it's not pass-by-value. Because

```
def append(list):
    list.append(1)

list = [0]
reassign(list)
append(list)
```

returns [0,1] showing that some kind of reference was clearly passed as pass-by-value does not allow a function to alter the parent scope at all.

Looks like pass-by-reference then, hu? Nope.

2. Here's why it's not pass-by-reference. Because

```
def reassign(list):
    list = [0, 1]

list = [0]
reassign(list)
print list
```

returns [0] showing that the original reference was destroyed when list was reassigned, pass-by-reference would have returned [0,1].

For more information look [here](#)

If you want your function to not manipulate outside scope, you need to make a copy of the input parameters that creates a new object.

```
from copy import copy

def append(list):
    list = copy(list)
    list.append(1)
    print list

list = [0]
append(list)
print list
```

Share Edit Follow Flag

edited Aug 26, 2019 at 1:54
Zain Arshad
1,905 1 12 26

answered Jul 22, 2015 at 18:21
Philip Dodson
337 3 9

Python already call by ref.

let's take example:

```
def foo(var):
    print(hex(id(var)))

x = 1 # any value
print(hex(id(x))) # I think the id() give the ref...
foo(x)
```

OutPut

```
0x08031708 Andth you might give another hex number depend on your memory
0x08031708
```

Share Edit Follow Delete Flag

answered Jun 6, 2017 at 22:12
Casper Ghost
5 1

No, it's not true: `def foo(var): print(id(var)) var = 1234 print(id(var)) x = 123 print(id(x)) # I think the id() give the ref...`
`foo(x) print(id(x))` – Zohrab Jan 25, 2017 at 10:13

Hope the following description sums it up well:

There are two things to consider here - variables and objects.

1. If you are passing a variable, then it's pass by value, which means the changes made to the variable within the function are local to that function and hence won't be reflected globally. This is more of a 'C' like behavior.

Example:

```
def changeval(myvar):
    myvar = 20
    print "values inside the function: ", myvar
    return

myvar = 10
changeval(myvar)
print "values outside the function: ", myvar
```

O/P:

values inside the function: 20
values outside the function: 10

2. If you are passing the variables packed inside a mutable object, like a list, then the changes made to the object are reflected globally as long as the object is not re-assigned.

Example:

```
def changelist(mylist):
    mylist.append("a")
    mylist.append(mylist)
    print "values inside the function: ", mylist
    return

mylist = [1,2,3]
changelist(mylist)
print "values outside the function: ", mylist
```

O/P:

values inside the function: [1, 2, 3, ['a']]
values outside the function: [1, 2, 3, ['a']]

3. Now consider the case where the object is re-assigned. In this case, the object refers to a new memory location which is local to the function in which this happens and hence not reflected globally.

Example:

```
def changelist(mylist):
    mylist=["a"]
    print "values inside the function: ", mylist
    return

mylist = [1,2,3]
changelist(mylist)
print "values outside the function: ", mylist
```

O/P:

values inside the function: ['a']
values outside the function: [1, 2, 3]

Share Edit Follow Flag

edited Jan 4, 2015 at 13:05
Ashique Kasurde
4,037 1 25 33

answered Jun 3, 2013 at 9:15
antboy192
2,069 3 22 28

2 This is the best summary! *** BEST SUMMARY HERE *** everyone read this answer! 1. pass simple variables 2. pass object reference which is modified 3. pass object reference which is reassigned in function – gaonthe Oct 25, 2017 at 10:22

5 This answer is very confused. Python doesn't pass different kinds of values differently; all function arguments in Python receive their values by assignment, and behave exactly the same way assignment behaves anywhere else in the language. ("Although that way may not be obvious at first unless you're Dutch.") – Daniel Pryden Mar 15, 2018 at 17:06

OK, I'll take a stab at this. Python passes by object reference, which is different from what you'd normally think of as "by reference" or "by value". Take this example:

```
def foo(x):
    print x

bar = 'some value'
foo(bar)
```

So you're creating a string object with value 'some value' and "binding" it to a variable named `bar`. In C, that would be similar to `bar` being a pointer to 'some value'.

When you call `foo(bar)`, you're not passing in `bar` itself. You're passing in `bar`'s value: a pointer to 'some value'. At that point, there are two "pointers" to the same string object.

Now compare that to:

```
def foo(x):
    x = 'another value'
    print x

bar = 'some value'
foo(bar)
```

Here's where the difference lies. In the line:

```
x = 'another value'
```

you're not actually altering the contents of `x`. In fact, that's not even possible. Instead, you're creating a new string object with value 'another value'. That assignment operation isn't saying "overwrite the thing `x` is pointing at with the new value". It's saying "update `x` to point at the new object instead". After that line, there are two string objects: 'some value' (with `bar` pointing at it) and 'another value' (with `x` pointing at it).

This isn't clumsy. When you understand how it works, it's a beautifully elegant, efficient system.

Share Edit Follow Flag

edited Sep 28, 2013 at 12:50
Rob Boykin
6 3 8

answered Nov 9, 2012 at 0:35
Kirk Strauser
30.5k 5 51 68

3 But what if I want the function to alter the contents of my variable? I take it that there is no way to do that? – aquardurte May 21, 2016 at 17:09

1 You can alter the object that the name is pointing at, e.g. by appending to a list. If that object is mutable. There are no Python semantics for saying "alter the namespace of the scope that called me so that the name in my argument list now points at something different than when it called me". – Kirk Strauser May 21, 2016 at 18:19

2 Effectively what I'm asking is if there's a simple way to get python not create a new pointer to the same object but to use the original pointer instead. I take it that there is no way to do this. I take it from other answers that the normal work around to achieving what passing by reference does is just to return more parameters. If that's the case, I see some validity in the argument that `abc,de = foo(a,b,c,d,e)` is a bit more clumsy than just `foo(a,b,c,d,e)`. – aquardurte May 21, 2016 at 19:13

2 @aquardurte This is nothing to do by the fact that a new pointer is created. You have to return a new object because `str`, `ints`, `tuple` & other objects are immutable. If you pass a mutable object, like a `list`, you can change it inside the function, without the need to return it, because all function parameters are pointers to the same objects you passed. – Marco Sula Jan 1, 2020 at 11:06

@aquardurte Python doesn't create a "new pointer". python doesn't have pointers! – jumpa.ariavillaga Nov 3, 2021 at 16:50

The answer given is

```
def set_4(x):
    y = []
    for i in x:
        y.append(i)
    return y
```

and

```
i = [0]

def set_3(x):
    x[i] = 3

set_3(i)
print(i[0])
```

which is the best answer so far as it does what it says in the question. However, it does seem a very clumsy way compared to VB or Pascal, is it the best method we have?

Not only is it clumsy, it involves mutating the original parameter in some way manually, eg by changing the original parameter to a list, or copying it to another list rather than just saying "use this parameter as a value" or "use this one as a reference". Could the simple answer be there is no reserved word for this but these are great work arounds?

Share Edit Follow Flag

edited Nov 9, 2012 at 0:01

answered Nov 8, 2012 at 23:53
Timothy Lawrens
2,244 6 25 34

So this is a little bit of a subtle point, because while Python only passes variables by value, every variable in Python is a reference. If you want to be able to change your values with a function call, what you need is a mutable object. For example:

1 - [0]

def set_3(x):

x[0] = 3

set_3([1])

print([1][0])

In the above code, the function modifies the contents of a List object (which is mutable), and so the output is 3 instead of 0.

I write this answer only to illustrate what "by value" means in Python. The above code is bad style, and if you really want to mutate your values you should write a class and call methods within that class, as MPX suggests.

Share Edit Follow Flag

answered Nov 8, 2012 at 23:13

Isaac

3,594

1

19

21

▲

this answers half the question so how would i write the same set_3(x) function so that it does not change the value of 1 but returns a new list that has been changed?

👤

Timothy Lawman Nov 8, 2012 at 23:31

▲

Just make a new list and modify it: y = [x for x in x]; y[0] = 3; return y → Isaac Nov 8, 2012 at 23:35

👤

▲

the whole thing involves lists and copying lists which goes in some ways to answer, but how can i do it with out having to convert an int to a list or is this the best we have?

👤

Timothy Lawman Nov 8, 2012 at 23:47

▲

You can't modify a primitive argument like an int. You have to put it in some sort of container. My example put it in a list. You could also put it in a class or a dictionary. Really though you should just reassign it outside a function with something like x = foo(x) → Isaac Nov 8, 2012 at 23:51

👤

-2

In Python the passing by reference or by value has to do with what are the actual objects you are passing. So if you are passing a list for example then you actually make this pass by reference, since the list is a mutable object. Thus you are passing a pointer to the function and you can modify the object (list) in the function body.

When you are passing a string this passing is done by value, so a new string object is being created and when the function terminates it is destroyed. So it all has to do with mutable and immutable objects.

Share Edit Follow Delete Flag

answered Nov 8, 2012 at 23:12

Julianus

1,137

1

8

11

1

▲

That's completely incorrect. Python always passes by "object reference" (which is different from variable reference) → Kirk Strauser Nov 9, 2012 at 0:20

👤

▲

i meant the analogy mutable objects-pass by reference and immutable objects-pass by value. It is perfect valid what i am saying. You probably did not catch what i was saying → Julianus Nov 9, 2012 at 20:56

👤

1

▲

i caught it and it's wrong. It has nothing to do with whether an object is mutable or immutable; it's always done the same way → Kirk Strauser Nov 9, 2012 at 21:31

👤

0

🔒

This post is hidden. It was deleted 4 years ago by the post author.

0

I understand the analogy. The analogy (in its effort to be concise) will actually confuse the newbies, I think. Or it could be explained in a better way. Here is my attempt:

In python, all variables are references-to-objects. So when you are passing variables to functions, you are actually passing the references-to-objects by value. Then there is this concept of immutable objects. Once created these objects can never be changed. So any reassignment to a variable (holding reference to an immutable object) will change the object being referenced (sometimes it will lead to a brand new object being created). This concept applies even within the same scope. I mean, you don't even have to pass variables to functions to observe this.

So when you pass an immutable object to a function, no new copy is created (remember: new copies are ALWAYS created in the case of pure pass-by-value). So it "behaves" like passing-by-reference as long as the object is not being changed. But the moment the variable is reassigned within the function, a new object is created (only because the object is immutable). The old immutable object will be quite intact though. When you return back from the function, we are referencing the old intact object. So in effect, it "behaves" like passing-by-value.

Share Edit Follow Undelete Flag

answered Jul 26, 2019 at 4:46

Bharu Sandaru

1

Comments disabled on deleted / locked posts / reviews

-1

🔒

This post is hidden. It was deleted 4 years ago by user3956566.

0

here is something i tried, i dont understand this my self though hope some one can explain this

<https://i.stack.imgur.com/vYj13.png>

Share Edit Follow Undelete Flag

answered Jul 16, 2019 at 16:11

user9580778

1

▲

This does not provide an answer to the question. Once you have sufficient reputation you will be able to comment on any post. Instead, provide answers that don't require clarification from the asker. - From Review - Error - Syntactical Remorse Jul 16, 2019 at 18:16

Comments disabled on deleted / locked posts / reviews

-3

🔒

This post is hidden. It was deleted 11 years ago by the post author.

0

I think you're talking about pointers. If that's the case, write a python class, not a function.

<http://docs.python.org/3/tutorial/classes.html>

Share Edit Follow Undelete Flag

answered Nov 8, 2012 at 23:07

MPX

24

6

43

1

5

1

▲

This has nothing to do with my question as i am talking about call by reference and value parameter arguments in a simple subroutine. → Timothy Lawman Nov 8, 2012 at 23:34

👤

▲

include me for trying to help. → MPX Nov 8, 2012 at 23:43

👤

Comments disabled on deleted / locked posts / reviews

https://stackoverflow.com/questions/13209427/python-functions-call-by-reference

33